

Tema 3 — Segurança

Usar IA para programar deixa o software mais inseguro?

O uso de inteligência artificial na programação pode ajudar bastante no desenvolvimento, principalmente pela velocidade com que ela consegue gerar código, explicar erros e sugerir soluções. Porém, isso também pode trazer alguns riscos de segurança, principalmente quando o desenvolvedor aceita as respostas sem revisar o que foi gerado.

O artigo principal, de Perry et al., analisou justamente esse problema. No estudo, pessoas que utilizaram um assistente de IA acabaram produzindo códigos menos seguros do que aquelas que não utilizaram. Além disso, os participantes que usaram IA também tinham uma tendência maior de acreditar que suas soluções estavam seguras. Isso mostra que um dos principais riscos não é apenas a IA gerar algo errado, mas o programador confiar demais na resposta.

Outro estudo, de Pearce et al., analisou códigos gerados pelo GitHub Copilot. Foram criados 89 cenários diferentes e 1.689 programas, sendo que aproximadamente 40% deles apresentavam algum tipo de vulnerabilidade. Isso mostra que um código pode funcionar normalmente e ainda assim possuir problemas de segurança.

De onde vêm as vulnerabilidades?

As vulnerabilidades de um software não vêm apenas de um lugar. Elas podem estar no próprio código, nas dependências utilizadas ou até na maneira como as pessoas trabalham com essas ferramentas.

No código, podem existir problemas como falta de validação dos dados, consultas SQL inseguras, erros de autenticação ou exposição de informações importantes.

Também existe o problema das dependências. O estudo de Spracklen et al. mostra que modelos de IA podem recomendar pacotes que na verdade nem existem. Esse problema é chamado de **package hallucination**, ou alucinação de pacotes.

Isso pode gerar um risco chamado **slopsquatting**. Nesse caso, um atacante pode registrar na internet um pacote com o mesmo nome que uma IA costuma inventar. Se posteriormente outro desenvolvedor receber essa sugestão da IA e instalar o pacote sem verificar sua origem, ele pode acabar instalando um código malicioso.

Por isso, podemos dizer que as vulnerabilidades podem vir dos três pontos: **código, dependências e pessoas**.

Quando um desenvolvedor confia numa sugestão, o que ele deixa de verificar?

Quando um desenvolvedor aceita uma sugestão da IA sem revisar, ele pode deixar de verificar se o código realmente é seguro, se existe alguma vulnerabilidade, se a biblioteca indicada realmente existe ou se aquela dependência é confiável.

Também pode deixar de analisar questões como validação de dados, controle de acesso, armazenamento de senhas, tokens e outras informações importantes.

Por isso, a IA deve servir como uma ferramenta de apoio. O desenvolvedor ainda precisa entender o código que está sendo colocado no sistema e verificar se aquela solução realmente faz sentido.

Velocidade e segurança caminham juntas?

Não necessariamente.

A IA consegue aumentar bastante a velocidade com que um código é produzido. O problema é que, se o desenvolvimento fica mais rápido e a revisão não acompanha essa velocidade, mais erros e vulnerabilidades podem acabar entrando no software.

Por exemplo, gerar várias funções em poucos segundos pode aumentar a produtividade, mas todas elas ainda precisam ser entendidas, testadas e revisadas.

Então, velocidade e segurança podem caminhar juntas, desde que existam revisão de código, testes e boas práticas de segurança. Caso contrário, produzir código mais rápido pode acabar significando também produzir problemas mais rápido.

Conclusão

Usar inteligência artificial para programar não deixa um software automaticamente inseguro. Porém, os estudos mostram que ela pode aumentar os riscos quando suas respostas são aceitas sem questionamento.

A IA pode gerar códigos vulneráveis, sugerir dependências inexistentes e até aumentar a confiança do desenvolvedor em uma solução que não é segura.

Por isso, o ideal é utilizar a IA como uma ferramenta para auxiliar e acelerar o desenvolvimento, mas sem deixar de revisar, testar e entender o código gerado. No final, a responsabilidade pela segurança do software continua sendo do desenvolvedor e da equipe que mantém o sistema.